



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Instrucciones:

1.- Terminar la Grúa con:

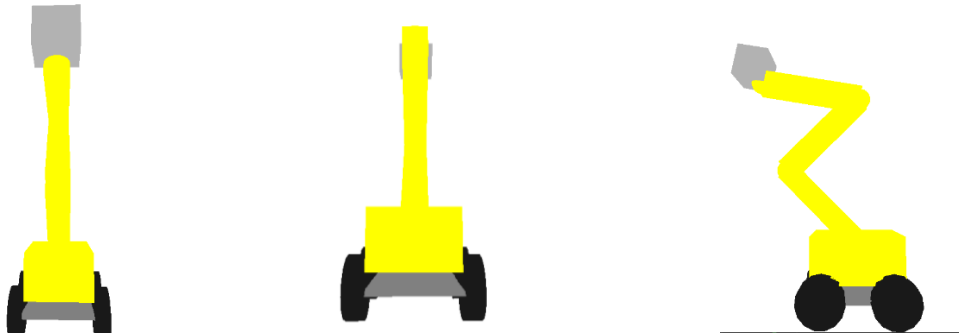
- cuerpo(prisma rectangular)
- base (pirámide cuadrangular)
- 4 llantas(4 cilindros) con teclado se pueden girar cada una de las 4 llantas por separado

2.- Crear un animal robot 3d

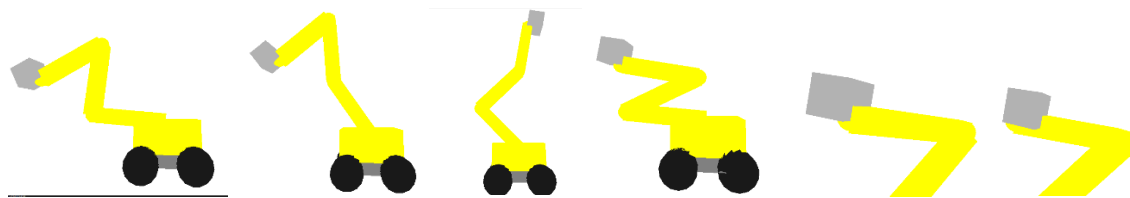
- Instanciando cubos, pirámides, cilindros, conos, esferas:
- 4 patas articuladas en 2 partes (con teclado se puede mover las dos articulaciones de cada pata)
- cola articulada o 2 orejas articuladas. (con teclado se puede mover la cola o cada oreja independiente

Capturas de pantalla del funcionamiento Actividad 1:

Capturas de pantalla de la ejecución:



Vista frontal, posterior y lateral.

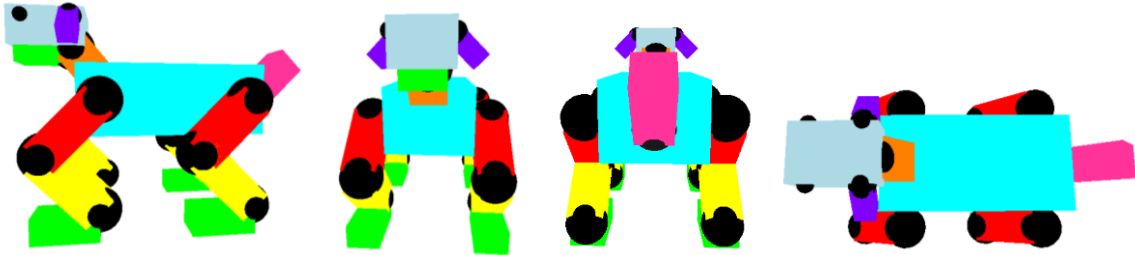


Vista diferentes movimientos de los brazos y canasta (en el eje y)

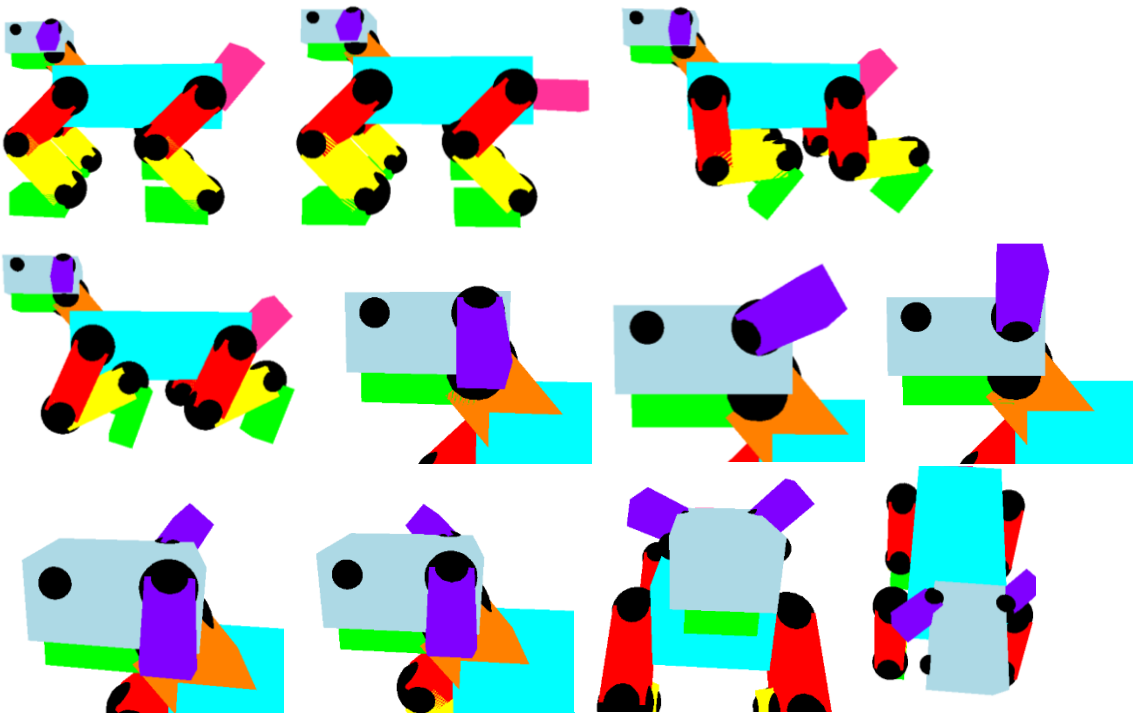
Nota: las capturas de pantalla de la rotación de las ruedas no aprecian correctamente el movimiento de las mismas no obstante estas corresponden a las teclas K, L, N, M para su rotación independiente

Capturas de pantalla del funcionamiento Actividad 2:

Capturas de pantalla de la ejecución:



Vista lateral, frontal, posterior y superior



Vista diferentes movimientos de cola, piernas (articulación 1 y 2) y orejas (articulaciones independientes)

Código generado:

Modificación window.cpp:

```
24 articulation5 = 0.0f;
25 articulation6 = 0.0f;
26 articulation7 = 0.0f; // Inicialización de nueva articulación
27 articulation8 = 0.0f; // Inicialización de nueva articulación
28
```

```
128 // Control Independiente de Llantas
129 if (key == GLFW_KEY_K) theWindow->articulation5 += 10.0f; // Llanta 1
130 if (key == GLFW_KEY_L) theWindow->articulation6 += 10.0f; // Llanta 2
131 if (key == GLFW_KEY_N) theWindow->articulation7 += 10.0f; // Llanta 3 (NUEVA)
132 if (key == GLFW_KEY_M) theWindow->articulation8 += 10.0f; // Llanta 4 (NUEVA)
133 }
```

Modificación window.h:

```
35 GLfloat getarticulation6() { return articulation6; }
36 GLfloat getarticulation7() { return articulation7; } // Nueva: Llanta 3
37 GLfloat getarticulation8() { return articulation8; } // Nueva: Llanta 4
38
```

```
44 // Variables de control de movimiento
45 GLfloat rotax, rotay, rotaz;
46 GLfloat articulation1, articulation2, articulation3, articulation4;
47 GLfloat articulation5, articulation6, articulation7, articulation8; // Añadidas 7 y 8
48
```

Modificación a las matrices de apoyo:

```
316 glm::mat4 model(1.0); // Inicializar matriz de Modelo 4x4
317
318 // Matrices de apoyo para la jerarquía del animal
319 glm::mat4 modelaux(1.0f); // Base para las patas
320 glm::mat4 colaPivote(1.0f); // Punto de conexión de la cola
321 glm::mat4 cabezaPivote(1.0f); // Punto de conexión del cuello/cabeza
322
323 glm::vec3 color = glm::vec3(0.0f, 0.0f, 0.0f); // Inicializar Color para enviar a variable Uniform;
324
```

Código de actividad 1:

Código de cabina:

```
352 // =====
353 // CABINA
354 // =====
355
356 // Reiniciar la matriz de modelo
357 model = glm::mat4(1.0);
358
359 // Traslación inicial para posicionar toda la grúa en el mundo
360 model = glm::translate(model, glm::vec3(0.0f, 2.0f, -4.0f));
361 // Guardamos esta transformación (posición global) para que los hijos la hereden
362 // pero sin la escala que aplicaremos a continuación.
363 modelaux = model;
364
365 // Transformaciones propias de la Cabina
366 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
367 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
368 color = glm::vec3(1.0f, 1.0f, 0.0f); // Amarillo
369 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
370 meshList[0]->RenderMesh(); // Dibuja el cubo de la cabina
```

Código de brazos articulados independientes:

```
372 // =====
373 // BRAZO 1
374 // =====
375 // Recuperamos la matriz de la cabina (solo posición, sin escala)
376 model = modelaux;
377
378 // articulación 1: Posicionamos el brazo respecto a la cabina (ej. en el techo)
379 model = glm::translate(model, glm::vec3(-0.5f, 0.2f, 0.0f));
380 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
381
382 // primer brazo que conecta con la cabina
383 // Aplicamos el desplazamiento para que rote desde un extremo
384 model = glm::translate(model, glm::vec3(-1.0f, 2.0f, 0.0f));
385 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
386 modelaux = model; // Guardamos para la articulación 2
387
388 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
389 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
390 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
391 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
392 color = glm::vec3(1.0f, 1.0f, 0.0f);
393 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
394 meshList[0]->RenderMesh();
```

```

396 // =====
397 // BRAZO 2
398 // =====
399 // Para descartar la escala cargamos la matrix auxiliar
400 model = modelaux;
401
402 // articulación 2
403 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
404 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
405 modelaux = model;
406
407 // dibujar una pequeña esfera
408 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
409 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
410 sp.render();
411
412 model = modelaux;
413 // segundo brazo
414 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
415 modelaux = model;
416
417 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
418 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
419 color = glm::vec3(1.0f, 1.0f, 0.0f);
420 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
421 meshList[0]->RenderMesh();
422
423 // =====
424 // BRAZO 3
425 // =====
426 // Para descartar la escala cargamos la matrix auxiliar del brazo 2
427 model = modelaux;
428
429 // articulación 3 (al final del brazo 2)
430 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
431 // Rotación de la articulación (controlada por teclado) + los 135 grados de la imagen
432 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3() + 35.0f), glm::vec3(0.0f, 0.0f, 1.0f));
433 modelaux = model;
434
435 // dibujar una pequeña esfera
436 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
437 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
438 sp.render();
439
440 model = modelaux;
441 // tercer brazo
442 // Se traslada en X porque, tras la rotación, el brazo se extiende "hacia adelante"
443 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
444 modelaux = model;
445
446 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
447 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
448 color = glm::vec3(1.0f, 1.0f, 0.0f);
449 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
450 meshList[0]->RenderMesh();

```

```

452 // Para descartar la escala cargamos la matrix auxiliar del brazo 3
453 model = modelaux;
454
455 // articulación 4 (punto final del brazo 3)
456 // Nos movemos al extremo del brazo 3 (que mide 5 unidades de largo)
457 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
458 // Rotación de la canasta (puedes usar una variable nueva o dejarla fija)
459 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 1.0f, 0.0f));
460 modelaux = model;
461
462 // dibujar una pequeña esfera (pivote de la canasta)
463 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
464 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
465 sp.render();

```

Código de la canasta:

```

468 // =====
469 // CANASTA
470 // =====
471 model = modelaux;
472 // La desplazamos un poco hacia abajo y hacia adelante para que cuelgue del pivote
473 model = glm::translate(model, glm::vec3(0.5f, -0.7f, 0.0f));
474 modelaux = model;
475
476 // Escalamos para que sea un cubo donde quepa una persona (
477 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.7f));
478 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
479 color = glm::vec3(0.7f, 0.7f, 0.7f); // Gris claro / Metálico
480 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
481 meshList[0]->RenderMesh(); // Dibuja el cubo
482

```

Código de la base de la cabina:

```

484 // =====
485 // BASE DE LA CABINA (Pirámide Cuadrangular)
486 // =====
487
488 // Reiniciamos a la matriz de la cabina (posición global)
489 // para no heredar rotaciones ni escalas de los brazos o la canasta.
490 model = glm::mat4(1.0);
491 model = glm::translate(model, glm::vec3(0.0f, 2.0f, -4.0f)); // Misma posición base que la cabina
492
493 // La movemos hacia abajo para que quede debajo del cubo amarillo
494 model = glm::translate(model, glm::vec3(0.0f, -0.7f, 0.0f));
495
496 // Guardamos para las llantas
497 modelaux = model;
498
499 // Escalamos la pirámide para que sea una base sólida
500 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.3f));
501
502 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
503 color = glm::vec3(0.5f, 0.5f, 0.5f); // Gris oscuro para la base
504 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
505
506 // Dibujamos la pirámide cuadrangular (índice 4 según tu código inicial)
507 meshList[4]->RenderMesh();

```

Códigos de las llantas (independientes)

```
509 // =====
510 // LLANTAS INDEPENDIENTES (COLOR NEGRO)
511 // =====
512
513 // --- LLANTA 1 ( - Tecla K) ---
514 model = modelaux;
515 model = glm::translate(model, glm::vec3(1.5f, -0.8f, 1.8f));
516 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
517 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
518 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 1.0f, 0.0f));
519 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
520 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
521 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.1f, 0.1f, 0.1f))); // Negro
522 meshList[2]->RenderMeshGeometry();
523
524 // --- LLANTA 2 ( - Tecla L) ---
525 model = modelaux;
526 model = glm::translate(model, glm::vec3(-1.5f, -0.8f, 1.8f));
527 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
528 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
529 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 1.0f, 0.0f));
530 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
531 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
532 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.1f, 0.1f, 0.1f))); // Negro
533 meshList[2]->RenderMeshGeometry();
534
```

```
535 // --- LLANTA 3 ( - Tecla N) ---
536 model = modelaux;
537 model = glm::translate(model, glm::vec3(1.5f, -0.8f, -1.8f));
538 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
539 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
540 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 1.0f, 0.0f));
541 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
542 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
543 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.1f, 0.1f, 0.1f))); // Negro
544 meshList[2]->RenderMeshGeometry();
545
546 // --- LLANTA 4 ( - Tecla M) ---
547 model = modelaux;
548 model = glm::translate(model, glm::vec3(-1.5f, -0.8f, -1.8f));
549 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
550 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
551 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 1.0f, 0.0f));
552 model = glm::scale(model, glm::vec3(1.0f, 0.6f, 1.0f));
553 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
554 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.1f, 0.1f, 0.1f))); // Negro
555 meshList[2]->RenderMeshGeometry();
```


Código de Actividad 2:

Código del torso:

```
561 // =====
562 // CUERPO DEL ANIMAL
563 // =====
564
565 // Reiniciar la matriz de modelo
566 model = glm::mat4(1.0);
567
568 // Posición en el mundo
569 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -7.0f));
570
571 // --- ALMACENAMIENTO DE MATRICES DE APOYO ---
572 modelaux = model; // Para las 4 patas
573 // Punto de salida para la cola
574 glm::mat4 colaPivote = glm::translate(model, glm::vec3(-2.0f, 0.4f, 0.0f));
575 // Punto de salida para el cuello/cabeza
576 glm::mat4 cabezaPivote = glm::translate(model, glm::vec3(2.0f, 0.4f, 0.0f));
577 // -----
578
579 // Escala del Torso
580 model = glm::scale(model, glm::vec3(4.0f, 1.5f, 2.0f));
581 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
582
583 // CAMBIO DE COLOR: Turquesa Brillante (R: 0.0, G: 1.0, B: 1.0)
584 color = glm::vec3(0.0f, 1.0f, 1.0f);
585 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
586
587 meshList[0]->RenderMesh(); // Dibuja el cubo del torso
588
```

Código de articulaciones (conexiones) a las patas:

```
589 // =====
590 // 1. ARTICULACIÓN DELANTERA DERECHA
591 // =====
592 model = modelaux; // Reset al cuerpo
593 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 1.35f));
594 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
595
596 // Dibujar Esfera Negra
597 glm::mat4 model_sph = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
598 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_sph));
599 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
600 sp.render();
601
602 // =====
603 // 2. ARTICULACIÓN DELANTERA IZQUIERDA
604 // =====
605 model = modelaux; // Reset al cuerpo
606 model = glm::translate(model, glm::vec3(1.5f, 0.0f, -1.35f)); // Z Negativa
607 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
608
609 // Dibujar Esfera Negra
610 model_sph = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
611 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_sph));
612 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
613 sp.render();
```

```

615 // =====
616 // 3. ARTICULACIÓN TRASERA DERECHA
617 // =====
618 model = modelaux; // Reset al cuerpo
619 model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 1.35f)); // X Negativa
620 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
621
622 // Dibujar Esfera Negra
623 model_sph = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
624 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_sph));
625 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
626 sp.render();
627
628 // =====
629 // 4. ARTICULACIÓN TRASERA IZQUIERDA
630 // =====
631 model = modelaux; // Reset al cuerpo
632 model = glm::translate(model, glm::vec3(-1.5f, 0.0f, -1.35f)); // X y Z Negativas
633 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
634
635 // Dibujar Esfera Negra
636 model_sph = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
637 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_sph));
638 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
639 sp.render();

```

Código de piernas:

```

641 // =====
642 // 1. MUSLO DELANTERO DERECHO (X: 1.5, Z: 1.35)
643 // =====
644 model = modelaux;
645 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 1.35f));
646 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
647 glm::mat4 pivote1 = model; // Punto de unión
648
649 // Dibujar Esfera y Muslo Rojo
650 glm::mat4 model_temp = glm::scale(pivote1, glm::vec3(0.45f, 0.45f, 0.45f));
651 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
652 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f))); // Negro
653 sp.render();
654
655 model_temp = glm::translate(pivote1, glm::vec3(0.75f, 0.0f, 0.0f));
656 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
657 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
658 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f))); // Rojo
659 meshList[0]->RenderMesh();

```

```

// =====
// 2. MUSLO DELANTERO IZQUIERDO (X: 1.5, Z: -1.35)
// =====
model = modelaux;
model = glm::translate(model, glm::vec3(1.5f, 0.0f, -1.35f));
model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glm::mat4 pivote2 = model;

// Dibujar Esfera y Muslo Rojo
model_temp = glm::scale(pivote2, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
sp.render();

model_temp = glm::translate(pivote2, glm::vec3(0.75f, 0.0f, 0.0f));
model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f)));
meshList[0]->RenderMesh();

```

```
// =====
// 3. MUSLO TRASERO DERECHO (X: -1.5, Z: 1.35)
// =====
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 1.35f));
model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glm::mat4 pivote3 = model;

// Dibujar Esfera y Muslo Rojo
model_temp = glm::scale(pivote3, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
sp.render();

model_temp = glm::translate(pivote3, glm::vec3(0.75f, 0.0f, 0.0f));
model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f)));
meshList[0]->RenderMesh();
```

```
701 // =====
702 // 4. MUSLO TRASERO IZQUIERDO (X: -1.5, Z: -1.35)
703 // =====
704 model = modelaux;
705 model = glm::translate(model, glm::vec3(-1.5f, 0.0f, -1.35f));
706 model = glm::rotate(model, glm::radians(-135.0f + mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
707 glm::mat4 pivote4 = model;
708
709 // Dibujar Esfera y Muslo Rojo
710 model_temp = glm::scale(pivote4, glm::vec3(0.45f, 0.45f, 0.45f));
711 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
712 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
713 sp.render();
714
715 model_temp = glm::translate(pivote4, glm::vec3(0.75f, 0.0f, 0.0f));
716 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
717 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
718 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.0f, 0.0f)));
719 meshList[0]->RenderMesh();
```

```
722 // =====
723 // 1. RODILLA DELANTERA DERECHA (Control: Articulación 2)
724 // =====
725 model = pivote1; // Salimos desde el hombro derecho
726 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f)); // Al final del muslo rojo
727 model = glm::rotate(model, glm::radians(90.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
728 glm::mat4 rodilla1 = model; // Nuevo pivote para la parte baja
729
730 // Esfera Rodilla 1
731 model_temp = glm::scale(rodilla1, glm::vec3(0.45f, 0.45f, 0.45f));
732 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
733 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
734 sp.render();
735
736 // Pantorrilla Amarilla 1
737 model_temp = glm::translate(rodilla1, glm::vec3(0.75f, 0.0f, 0.0f));
738 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
739 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
740 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 1.0f, 0.0f)));
741 meshList[0]->RenderMesh();
742
```

```

743 // =====
744 // 2. RODILLA DELANTERA IZQUIERDA
745 // =====
746 model = pivote2;
747 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
748 model = glm::rotate(model, glm::radians(90.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
749 glm::mat4 rodilla2 = model;
750
751 // Esfera Rodilla 2
752 model_temp = glm::scale(rodilla2, glm::vec3(0.45f, 0.45f, 0.45f));
753 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
754 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
755 sp.render();
756
757 // Pantorrilla Amarilla 2
758 model_temp = glm::translate(rodilla2, glm::vec3(0.75f, 0.0f, 0.0f));
759 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
760 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
761 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 1.0f, 0.0f)));
762 meshList[0]->RenderMesh();
763

```

```

764 // =====
765 // 3. RODILLA TRASERA DERECHA
766 // =====
767 model = pivote3;
768 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
769 model = glm::rotate(model, glm::radians(90.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
770 glm::mat4 rodilla3 = model;
771
772 // Esfera Rodilla 3
773 model_temp = glm::scale(rodilla3, glm::vec3(0.45f, 0.45f, 0.45f));
774 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
775 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
776 sp.render();
777
778 // Pantorrilla Amarilla 3
779 model_temp = glm::translate(rodilla3, glm::vec3(0.75f, 0.0f, 0.0f));
780 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
781 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
782 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 1.0f, 0.0f)));
783 meshList[0]->RenderMesh();

```

```

785 // =====
786 // 4. RODILLA TRASERA IZQUIERDA
787 // =====
788 model = pivote4;
789 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
790 model = glm::rotate(model, glm::radians(90.0f + mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
791 glm::mat4 rodilla4 = model;
792
793 // Esfera Rodilla 4
794 model_temp = glm::scale(rodilla4, glm::vec3(0.45f, 0.45f, 0.45f));
795 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
796 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
797 sp.render();
798
799 // Pantorrilla Amarilla 4
800 model_temp = glm::translate(rodilla4, glm::vec3(0.75f, 0.0f, 0.0f));
801 model_temp = glm::scale(model_temp, glm::vec3(1.5f, 0.7f, 0.7f));
802 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
803 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 1.0f, 0.0f)));
804 meshList[0]->RenderMesh();
805

```

```

806 // =====
807 // 1. PIE DELANTERO DERECHO (Fijo -135°)
808 // =====
809 model = rodilla1; // Salimos desde la rodilla derecha
810 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f)); // Al final de la pantorrilla amarilla
811 model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
812 glm::mat4 pie1 = model;
813
814 // Esfera Tobillo 1
815 model_temp = glm::scale(pie1, glm::vec3(0.42f, 0.42f, 0.42f));
816 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
817 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
818 sp.render();
819
820 // Pezuña Verde 1
821 model_temp = glm::translate(pie1, glm::vec3(0.625f, 0.25f, 0.0f)); // 0.625 es la mitad de 1.25
822 model_temp = glm::scale(model_temp, glm::vec3(1.25f, 0.5f, 0.7f));
823 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
824 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
825 meshList[0]->RenderMesh();
826

```

```

827 // =====
828 // 2. PIE DELANTERO IZQUIERDO
829 // =====
830 model = rodilla2;
831 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
832 model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
833 glm::mat4 pie2 = model;
834
835 // Esfera Tobillo 2
836 model_temp = glm::scale(pie2, glm::vec3(0.42f, 0.42f, 0.42f));
837 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
838 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
839 sp.render();
840
841 // Pezuña Verde 2
842 model_temp = glm::translate(pie2, glm::vec3(0.625f, 0.25f, 0.0f));
843 model_temp = glm::scale(model_temp, glm::vec3(1.25f, 0.5f, 0.7f));
844 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
845 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
846 meshList[0]->RenderMesh();
847

```

```

848 // =====
849 // 3. PIE TRASERO DERECHO
850 // =====
851 model = rodilla3;
852 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
853 model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
854 glm::mat4 pie3 = model;
855
856 // Esfera Tobillo 3
857 model_temp = glm::scale(pie3, glm::vec3(0.42f, 0.42f, 0.42f));
858 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
859 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
860 sp.render();
861
862 // Pezuña Verde 3
863 model_temp = glm::translate(pie3, glm::vec3(0.625f, 0.25f, 0.0f));
864 model_temp = glm::scale(model_temp, glm::vec3(1.25f, 0.5f, 0.7f));
865 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
866 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
867 meshList[0]->RenderMesh();
868

```

```

869 // =====
870 // 4. PIE TRASERO IZQUIERDO
871 // =====
872 model = rodilla4;
873 model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
874 model = glm::rotate(model, glm::radians(-135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
875 glm::mat4 pie4 = model;
876
877 // Esfera Tobillo 4
878 model_temp = glm::scale(pie4, glm::vec3(0.42f, 0.42f, 0.42f));
879 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
880 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
881 sp.render();
882
883 // Pezuña Verde 4
884 model_temp = glm::translate(pie4, glm::vec3(0.625f, 0.25f, 0.0f));
885 model_temp = glm::scale(model_temp, glm::vec3(1.25f, 0.5f, 0.7f));
886 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model_temp));
887 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f)));
888 meshList[0]->RenderMesh();

```

Código de la cola del perro:

```

890 // =====
891 // ARTICULACIÓN DE LA COLA (Articulación 3 )
892 // =====
893 model = colaPivote; // Iniciamos desde el anclaje trasero del cuerpo
894
895 // 1. Posicionamos la articulación en el extremo superior de la cola
896 model = glm::translate(model, glm::vec3(4.0f, -0.5f, 0.0f));
897
898 // 2. Rotación dinámica controlada por el usuario
899 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(1.0f, 0.5f, 0.5f));
900
901 // 3. Inclinación fija de la cola (45 grados sobre el eje Z)
902 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
903
904 // Guardamos la matriz resultante como la base jerárquica
905 glm::mat4 articulacionCola = model;
906
907 // 4. Dibujar Esfera de la articulación (Mantiene su color negro para contraste)
908 model = glm::scale(articulacionCola, glm::vec3(0.4f, 0.4f, 0.4f));
909 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
910 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.1f, 0.1f, 0.1f)));
911 sp.render();
912
913 // 5. Dibujar el segmento físico de la cola
914 model = articulacionCola;
915 model = glm::translate(model, glm::vec3(0.75f, 0.0f, 0.0f));
916 model = glm::scale(model, glm::vec3(1.5f, 0.7f, 0.7f));
917
918 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
919
920 // CAMBIO DE COLOR: Rosa Brillante (R: 1.0, G: 0.2, B: 0.6)
921 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.2f, 0.6f)));
922
923 meshList[0]->RenderMesh();
924

```


Código del cuello:

```
926 // =====
927 // CUELLO (Segmento fijo y unión para cabeza)
928 // =====
929 model = cabezaPivote; // Iniciamos desde el anclaje delantero del cuerpo
930
931 // 1. Posicionamos el origen del cuello relativo al anclaje del cuerpo
932 model = glm::translate(model, glm::vec3(-3.5f, 0.0f, 0.0f));
933
934 // 2. Inclinación fija del cuello (130 grados para orientación ascendente)
935 model = glm::rotate(model, glm::radians(130.0f), glm::vec3(0.0f, 0.0f, 1.0f));
936
937 // Guardamos esta matriz para heredar la posición y rotación a la cabeza
938 glm::mat4 cuelloBase = model;
939
940 // 3. Dibujar el segmento del cuello
941 model = cuelloBase;
942 // Trasladamos 0.5f (mitad del largo) para que el cubo se extienda desde el origen
943 model = glm::translate(model, glm::vec3(0.5f, 0.0f, 0.0f));
944 // Escala: Largo de 1.0 y grosor de 0.8
945 model = glm::scale(model, glm::vec3(1.0f, 0.8f, 0.8f));
946
947 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
948
949 // CAMBIO DE COLOR: Naranja Brillante (R: 1.0, G: 0.5, B: 0.0)
950 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f, 0.5f, 0.0f)));
951
952 meshList[0]->RenderMesh();
953
954 // 4. ARTICULACION FINAL CUELLO (Esfera negra para conectar con la cabeza)
955 model = cuelloBase;
956 // Nos movemos al extremo final del cuello (largo total de 1.0f)
957 model = glm::translate(model, glm::vec3(1.0f, 0.0f, 0.0f));
958
959 // Guardamos esta matriz para que la cabeza nazca desde este punto
960 glm::mat4 articulacionCabezaPivote = model;
961
962 // Dibujo de la esfera de unión (Se mantiene negra para resaltar la articulación)
963 model = glm::scale(articulacionCabezaPivote, glm::vec3(0.5f, 0.5f, 0.5f));
964 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
965 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));
966 sp.render();
```

Código de la cabeza del perro:

```
969 // =====
970 // CABEZA - PARTE 1: MANDÍBULA (Verde)
971 // =====
972 model = articulacionCabezaPivote; // Nace del extremo final del cuello
973
974 // 1. Rotación fija de la mandíbula (50 grados en Z)
975 model = glm::rotate(model, glm::radians(50.0f), glm::vec3(0.0f, 0.0f, 1.0f));
976
977 // Guardamos este estado como base para la jerarquía de la boca
978 glm::mat4 mandibulaBase = model;
979
980 // 2. Dibujar segmento de la mandíbula
981 model = glm::translate(mandibulaBase, glm::vec3(0.6f, 0.0f, 0.0f)); // 0.6 es la mitad del largo 1.2
982 model = glm::scale(model, glm::vec3(1.2f, 0.7f, 0.8f));
983
984 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
985 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 1.0f, 0.0f))); // Verde identificación
986 meshList[0]->RenderMesh();
987 // =====
988 // CABEZA - PARTE 2: CRÁNEO (Más grande que la mandíbula)
989 // =====
990 model = mandibulaBase;
991
992 // 1. Posicionamiento del Cráneo (ajustado para encajar sobre la mandíbula)
993 model = glm::translate(model, glm::vec3(0.5f, -0.4f, 0.0f));
994 glm::mat4 craneoBase = model; // Matriz de referencia para el cráneo y sus hijos
995
996 // 2. Dibujar segmento del Cráneo
997 model = glm::scale(craneoBase, glm::vec3(1.6f, 0.8f, 1.1f));
998 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
999
1000 // CAMBIO DE COLOR: Azul Pastel (R: 0.68, G: 0.85, B: 0.9)
1001 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.68f, 0.85f, 0.9f)));
1002
1003 meshList[0]->RenderMesh();
```

```
1005 // =====
1006 // ARTICULACIONES DE LAS OREJAS (Esferas)
1007 // =====
1008
1009 // --- Esfera Oreja Derecha ---
1010 // Posicionamos el pivote en la parte superior y lateral del cráneo
1011 model = glm::translate(craneoBase, glm::vec3(-0.5f, -0.2f, 0.55f));
1012 glm::mat4 articulacionOrejaD = model; // Guardamos para la oreja derecha
1013
1014 // Dibujo de la esfera pequeña (escala 0.25 para que sea proporcional)
1015 model = glm::scale(articulacionOrejaD, glm::vec3(0.25f, 0.25f, 0.25f));
1016 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1017 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f))); // Negro
1018 sp.render();
1019
1020 // --- Esfera Oreja Izquierda ---
1021 // Posicionamos el pivote en el lado opuesto del eje Z
1022 model = glm::translate(craneoBase, glm::vec3(-0.5f, -0.2f, -0.55f));
1023 glm::mat4 articulacionOrejaI = model; // Guardamos para la oreja izquierda
1024
1025 // Dibujo de la esfera pequeña
1026 model = glm::scale(articulacionOrejaI, glm::vec3(0.25f, 0.25f, 0.25f));
1027 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1028 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f))); // Negro
1029 sp.render();
1030
```


Código de las orejas (movimiento independiente):

```
1031 // =====
1032 // OREJAS - COLOR MORADO ELÉCTRICO (Movimiento Independiente)
1033 // =====
1034
1035 // --- OREJA DERECHA ---
1036 model = articulacionOrejaD;
1037 // Rotación fija e inclinación dinámica con Articulación 4
1038 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1039 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 1.0f, 1.0f));
1040 glm::mat4 orejaDPivot = model;
1041
1042 // Dibujo Oreja Derecha
1043 model = glm::translate(orejaDPivot, glm::vec3(0.0f, 0.35f, 0.0f));
1044 model = glm::scale(model, glm::vec3(0.4f, 0.7f, 0.4f));
1045 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1046
1047 // CAMBIO DE COLOR: Morado Eléctrico (R: 0.5, G: 0.0, B: 1.0)
1048 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.5f, 0.0f, 1.0f)));
1049 meshList[0]->RenderMesh();
1050
1051 // --- OREJA IZQUIERDA ---
1052 model = articulacionOrejaI;
1053 // Rotación fija e inclinación dinámica con Articulación 5
1054 model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1055 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, -1.0f, 1.0f));
1056 glm::mat4 orejaIPivot = model;
1057
1058 // Dibujo Oreja Izquierda
1059 model = glm::translate(orejaIPivot, glm::vec3(0.0f, 0.35f, 0.0f));
1060 model = glm::scale(model, glm::vec3(0.4f, 0.7f, 0.4f));
1061 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1062
1063 // CAMBIO DE COLOR: Morado Eléctrico
1064 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.5f, 0.0f, 1.0f)));
1065 meshList[0]->RenderMesh();
```

Código ojos del perro (elemento decorativo):

```
1067 // =====
1068 // OJOS (Esferas Negras)
1069 // =====
1070 // Ojo Derecho
1071 model = glm::translate(craneoBase, glm::vec3(0.5f, -0.2f, 0.55f));
1072 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f)); // Pequeños y discretos
1073 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1074 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f))); // Negro
1075 sp.render();
1076
1077 // Ojo Izquierdo
1078 model = glm::translate(craneoBase, glm::vec3(0.5f, -0.2f, -0.55f));
1079 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));
1080 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1081 glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f))); // Negro
1082 sp.render();
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Actividad errores/problemas:

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

En esta ocasión la actividad 1 y 2 no me representaron ningún problema no obstante respecto a la actividad 2 el manejar articulaciones y valores heredados en un inicio me ocasionaron un poco de problemas ya que se producían rotaciones o traslaciones inesperadas, por lo que en lugar de hacer cada pata/articulación por separado fui haciendo en un por etapas las 4 articulaciones, brazos, y elementos para ir manejando las correspondientes matrices y desplazamientos para su manejo

Y respecto a la actividad 1 la única “problemática” a la que me enfrente fue el manejo de las 4 llantas ya que en momentos por los colores casi no podía identificar el movimiento de las mismas.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En referencia a detalles de la explicación teórica curiosamente me costo seguir el ritmo de la clase, pero cuando aprecie los videos y materiales proporcionados por el profesor me permitió realizar correctamente los ejercicios de clase y la práctica.

c. Conclusión de la práctica:

De esta práctica puedo concluir que el manejo de escalados y traslaciones entre modelos requiere cuidado ya que si no se maneja de manera correcta puede producir rotaciones o traslaciones incorrectas, en adición, esta práctica me enseñó que la jerarquía entre los elementos es lo que realmente da vida al modelo; sin ese orden, las piezas se moverían sin sentido por el espacio. Aprendí que el manejo de rotaciones y traslaciones no es solo aplicar fórmulas, sino entender cómo una pieza afecta a la siguiente.

Aunque al principio tuve algunos problemas para que las articulaciones rotaran desde donde yo quería, me sirvió mucho ir por partes. Decidí no desesperarme y armar primero todas las patas, luego la cola y dejar al final las orejas. Esto me permitió detectar errores más rápido. Además, el uso de matrices de apoyo fue un "salvavidas" para la actividad 2, ya que me permitieron conectar todo al cuerpo principal sin que las escalas de los cubos deformaran el resto de las extremidades. Al final, ver que cada llanta o cada oreja responde de forma independiente a mis comandos fue muy satisfactorio."

Bibliografía

- De Vries, J. (s.f.). Learn OpenGL: Transformations. Recuperado el 10 de marzo de 2026, de <https://learnopengl.com/Getting-started/Transformations>
- G-Truc Creation. (s.f.). GLM API documentation: Matrix transformations. Recuperado el 10 de marzo de 2026, de <https://glm.g-truc.net/0.9.9/api/a00665.html>
- Stack Overflow. (2015, 23 de abril). Proper way to handle hierarchical transformations in OpenGL. <https://stackoverflow.com/questions/29828551/proper-way-to-handle-hierarchical-transformations-in-opengl>
- Wikipedia contributors. (2026, 15 de febrero). Transformation matrix. Wikipedia. https://en.wikipedia.org/wiki/Transformation_matrix